



Noah Laghlali  
Emma Klingler  
Hugo Rochedreux





# The Silent Hunt



Noah Laghlali  
Emma Klingler  
Hugo Rochedreux

07/04

# Sommaire

- 1- Objectifs de l'itération**
- 2- Débogages**
- 3- Finitions UX / UI**
- 4- Behaviour Tree Lapin bot**
- 5- Bilan**



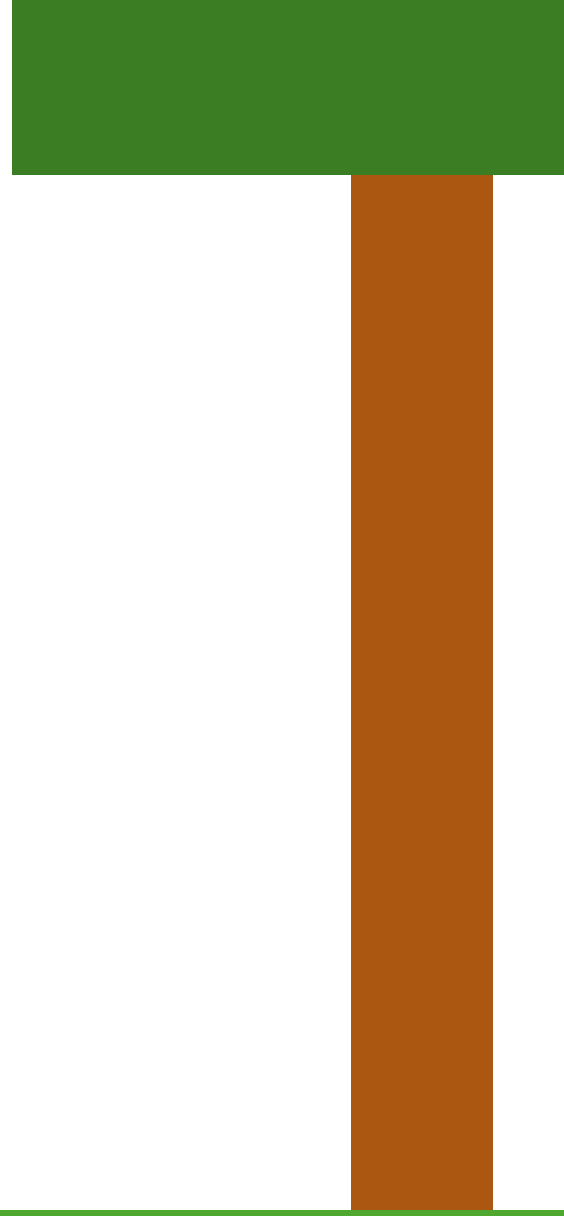
# Objectifs itération V

- Régler les derniers bugs en général
- Ajouter des utility sur les actions restantes du chasseur
- Finitions UX / UI
- Finalisation du Behaviour tree du Lapin Bot



# Débogages:

- Le chasseur gère plusieurs lapins
- Il n'y a plus de bug de connexion



# UX / UI :

- Ajout d'une carte miniature
- Ajout d'une barre de temps
- Ajout du compteur de joueurs
- Finalisation de la partie tuto
- Cycle jour / nuit
- Ajout Bordure de la carte



# UX / UI :

- Détection du son des lapins
- Jouable Mobile
- Ajout de règles du jeu



# Début Utility :

- Actions : Ravitaillement, Poursuite, Patrouille
- Sous estimation du temps
- Conclusion : Fonctions très simples





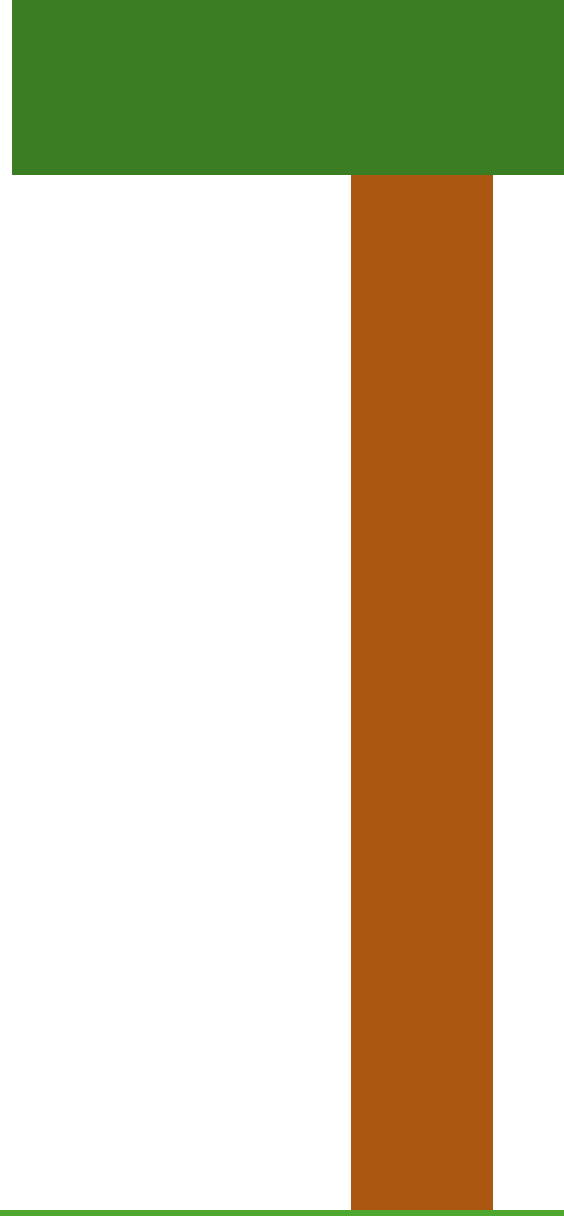
# Fonction Utility :

| Action         | Conditions principales             |
|----------------|------------------------------------|
| Poursuite      | énergie + vision + munitions       |
| Ravitaillement | manque de munitions + pas de cible |
| Exploration    |                                    |



# Behaviour Tree du Lapin robot :

- Fuite intelligente du chasseur
- Recherche et consommation de carottes
- Exploration aléatoire de la carte
- Émission de bruits (détectable par le chasseur)

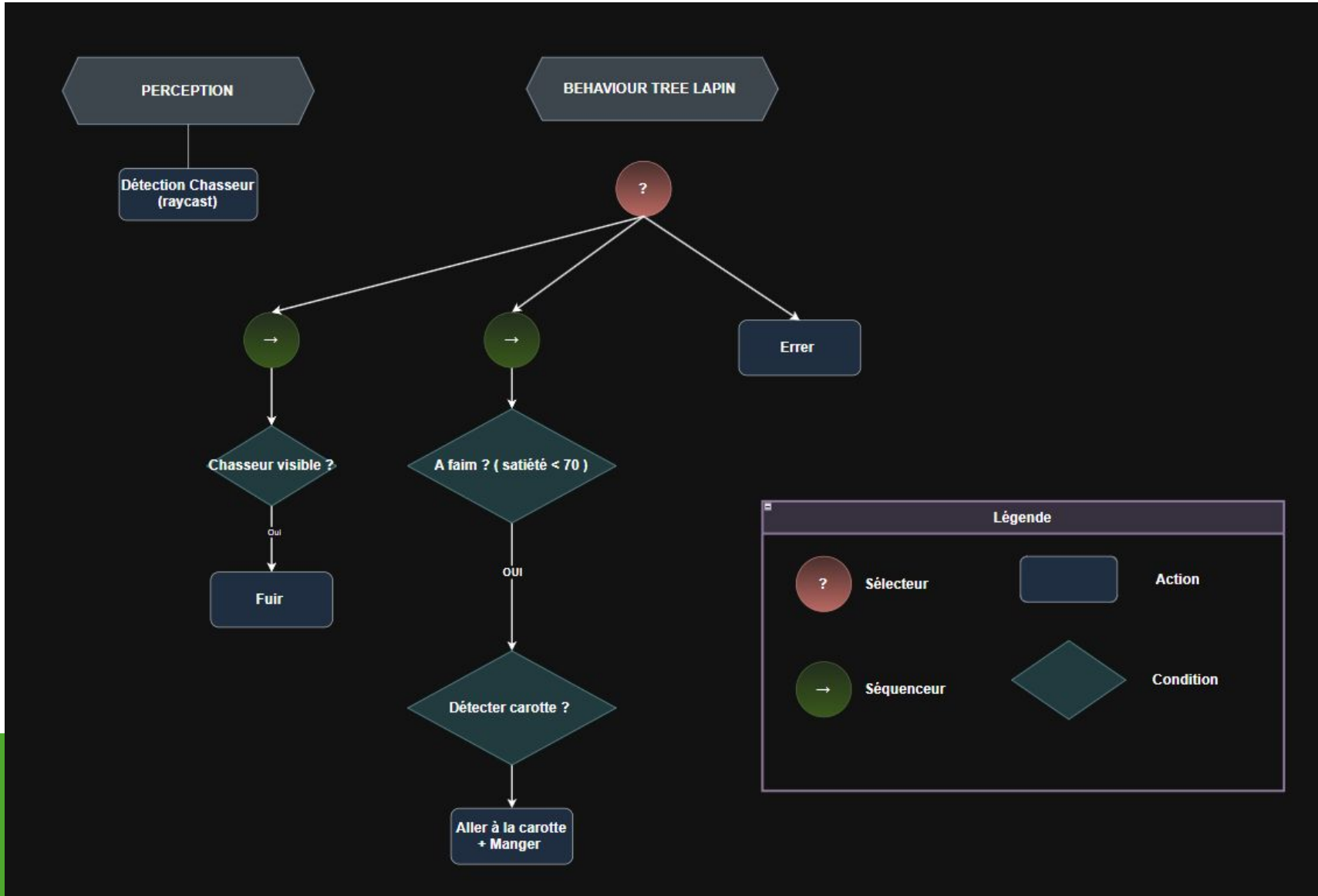


# Behaviour Tree du Lapin robot :

- Behaviour Tree identique au chasseur (Selector + Sequence)
- Animations synchronisées avec les lapins joueurs
- Intégré au système de détection du chasseur
- Connecté au lobby et à l'initialisation de la partie



# Behaviour Tree du Lapin robot :



# Répartition des tâches

| Hugo                            | Emma         | Noah                    |
|---------------------------------|--------------|-------------------------|
| Débogages                       | BT Bot Lapin | Système cycle jour/nuit |
| Finitions UX/UI                 |              |                         |
| Début Utility actions restantes |              |                         |

# Objectifs Itération VI :

- Rédaction du rapport final
- Rendre l'IA plus performante
- Derniers débogages



# Difficultés rencontrées

- Perte d'un équipier
- Fonction d'utilité simplifiées
- Attributs du chasseur non dynamiques



# Conclusion

- Chasseur et bots opérationnels
- Architecture Behaviour Tree cohérente et extensible
- Base solide pour de futures améliorations







# Démonstration

Joueurs restants : 1





# Présentation



Noah Laghlali  
Emma Klingler  
Hugo

07/04

# Sommaire

- Organisation
- Intelligence artificielle
- Architecture et Communication
- Outils et Environnement

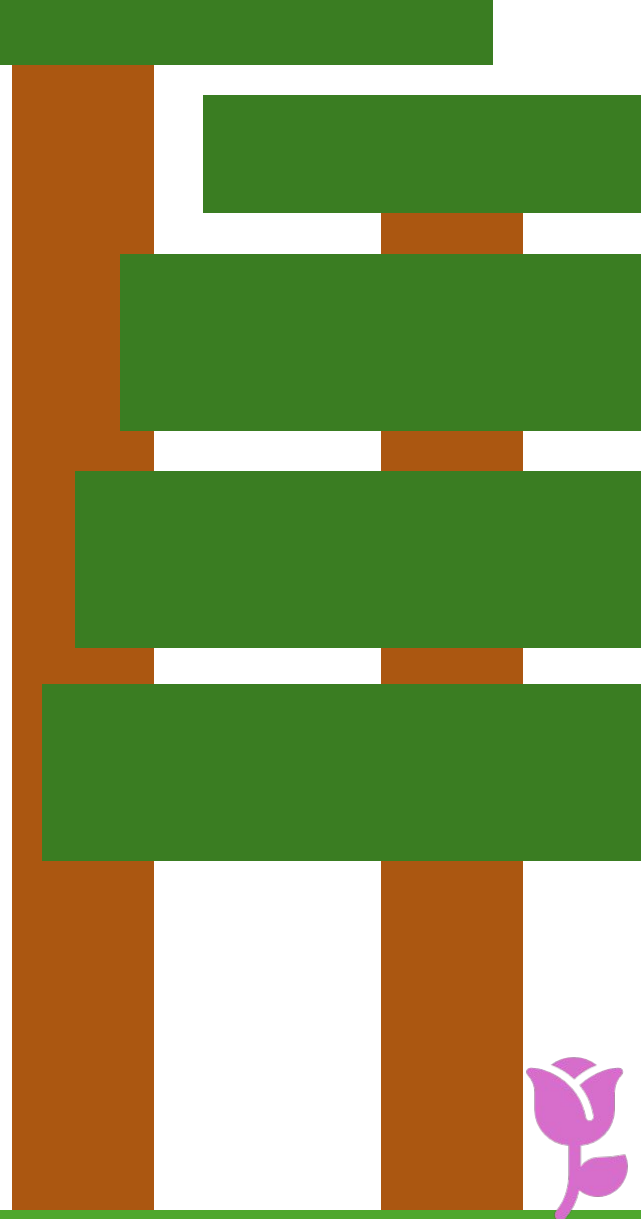
# Organisation

- Projet tutoré de fin de 3e année de BUT
- Travail collaboratif par équipe de 2.5 en méthode agile
- 204 heures de projet et 5 itérations
- Rendez-vous réguliers avec notre tuteur



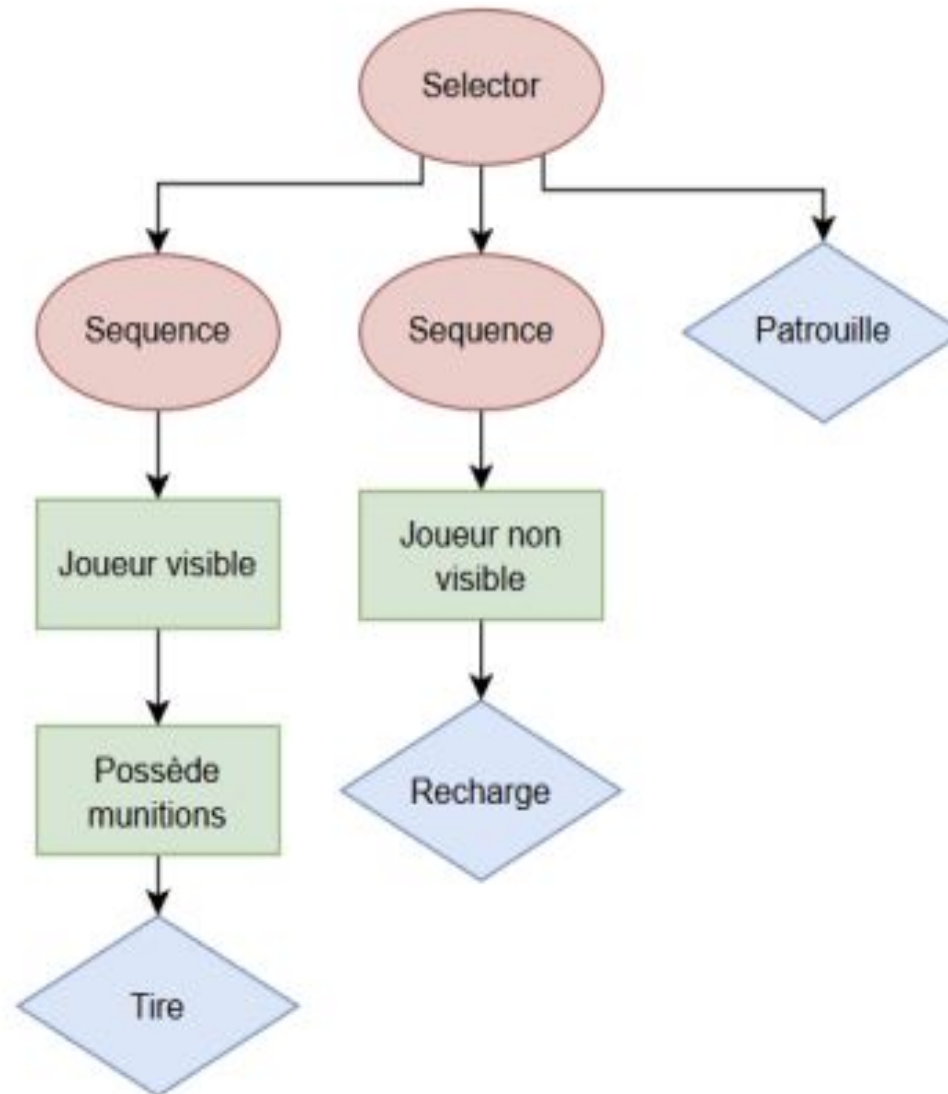
# Intelligence artificielle

- Behaviour Tree ( Selector, Sequence, conditions actions )
- Utility AI → poids dynamiques pour prioriser les actions
- Pathfinding
- Raycasting





# Exemple de Behaviour tree simple

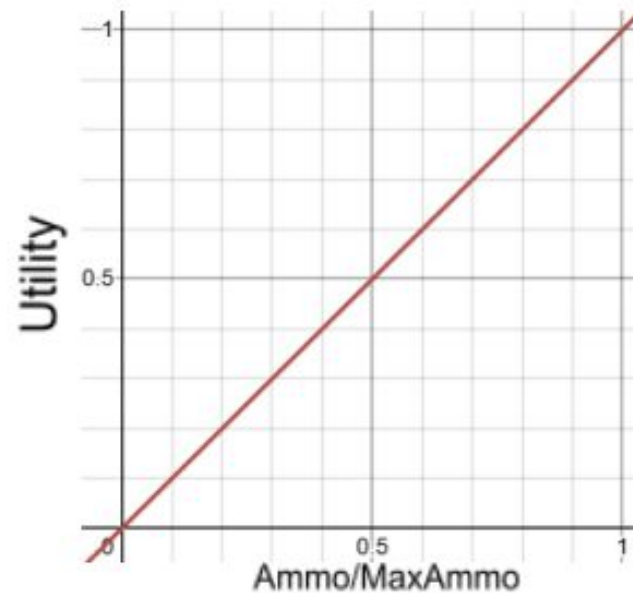


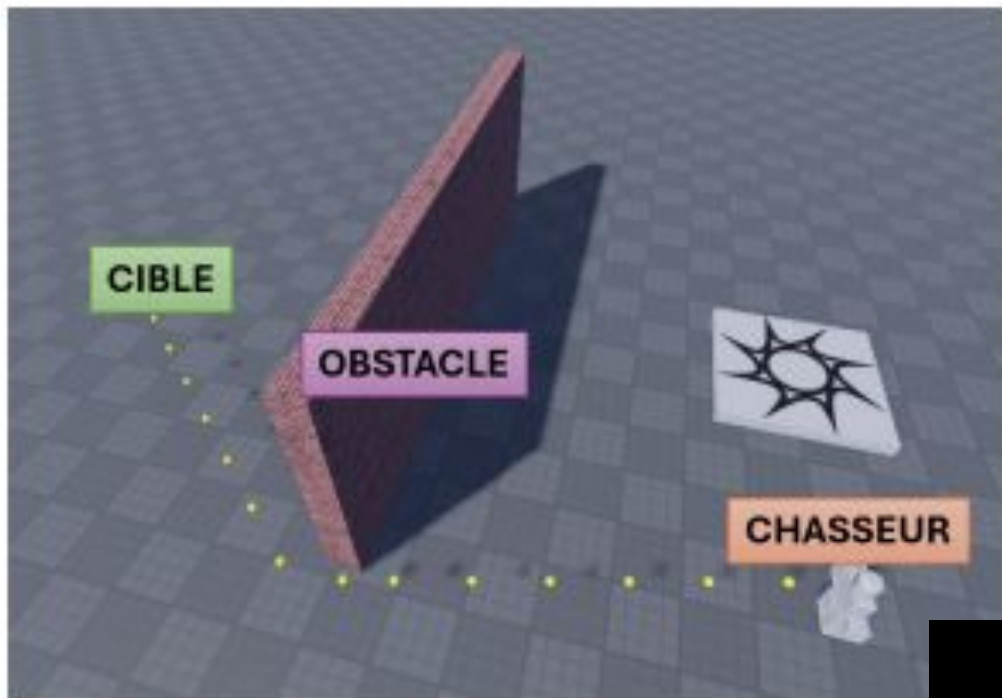
# Exemple de calcul d'Utility

- **Ammo** : nombre de balles dans le fusil du chasseur
- **MaxAmmo** : nombre de balles maximum dans le fusil du chasseur

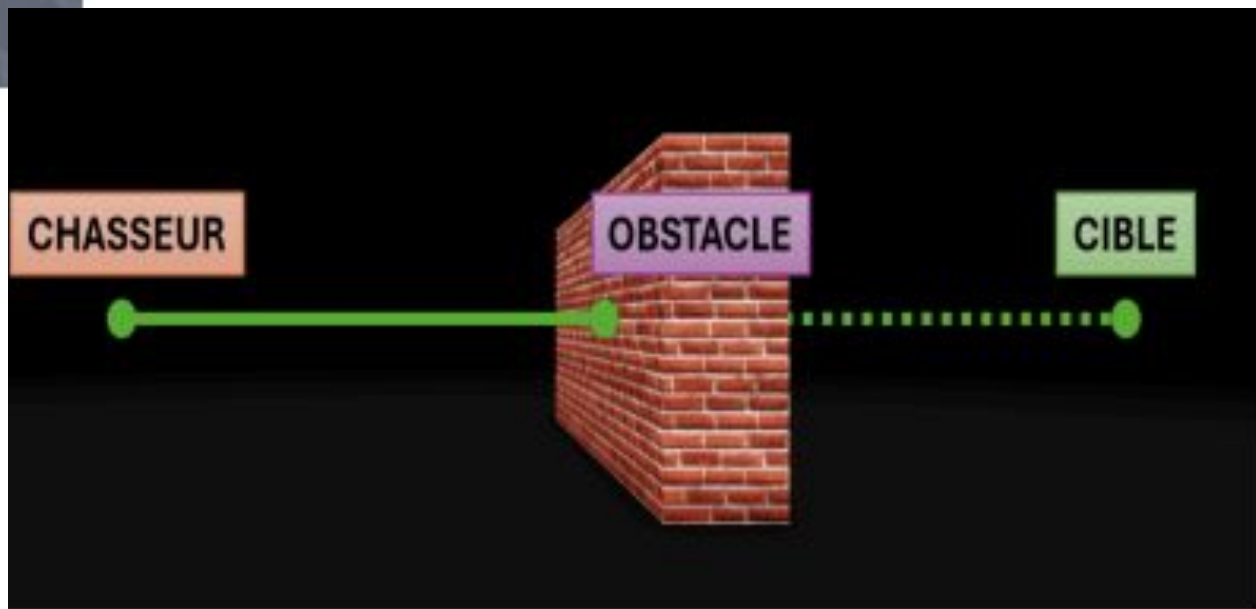
On pourrait alors créer une formule simple :

$$Utility(Recharge) = 1 - (Ammo/MaxAmmo)$$





Pathfinding



Raycasting

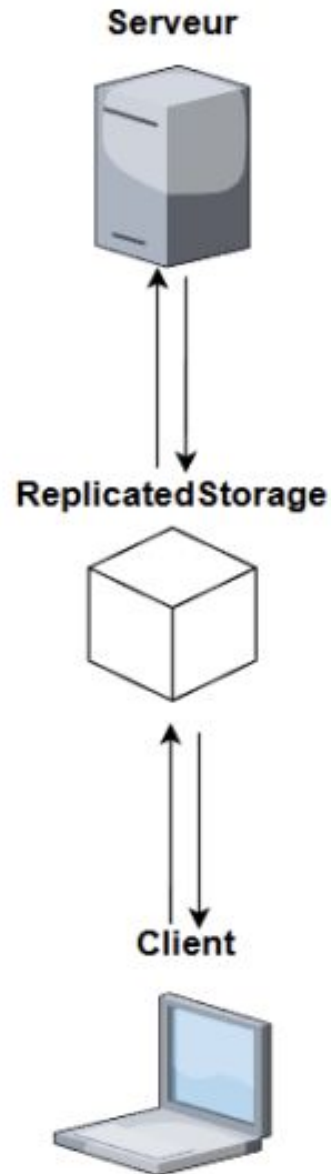


# Architecture et Communication

- Architecture client/serveur sur Roblox
- RemoteEvents → synchronisation et états entre serveur et clients



# Communication



**ReplicatedStorage** = espace partagé entre le serveur et tous les clients.

**ReplicatedStorage** contient :

- RemoteEvent
- RemoteFunction
- Modules partagés
- Assets communs

## RemoteEvent

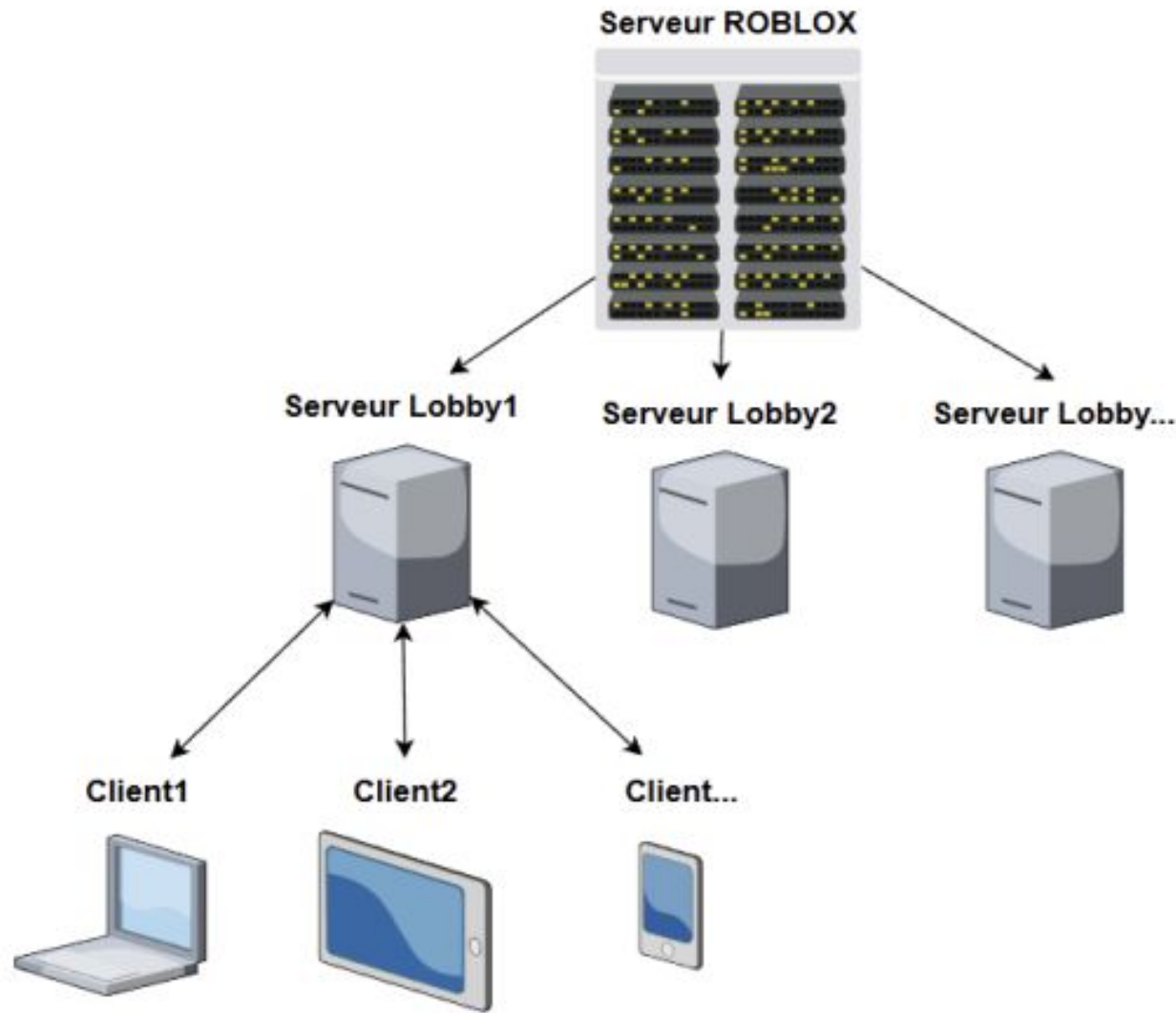
Communication asynchrone  
→ Pas de retour direct

Le client appelle une remote, le serveur l'écoute et traite la requête

### **Ou l'inverse**

Le serveur appelle une remote, le client destiné l'écoute et traite la requête

# Architecture Globale



## Roblox gère quoi

- Hébergement des serveurs
- Création automatique des Game Servers
- Load balancing
- Authentification des joueurs
- DataStore (sauvegarde des données)

## On gère quoi

### Côté Serveur

- Scripts serveur (Code Serveur)
- Logique du jeu
- Gestion des joueurs
- Synchronisation des états

### Côté Client

- LocalScripts (Code Client)
- Interface utilisateur (GUI)
- Effets visuels, animations
- Sons
- Inputs joueur

# Outils et Environnement

- Roblox Studio / Luau
- Vs Code + Rojo
- Github
- Trello







Joueurs restants : 1

# Démonstration

